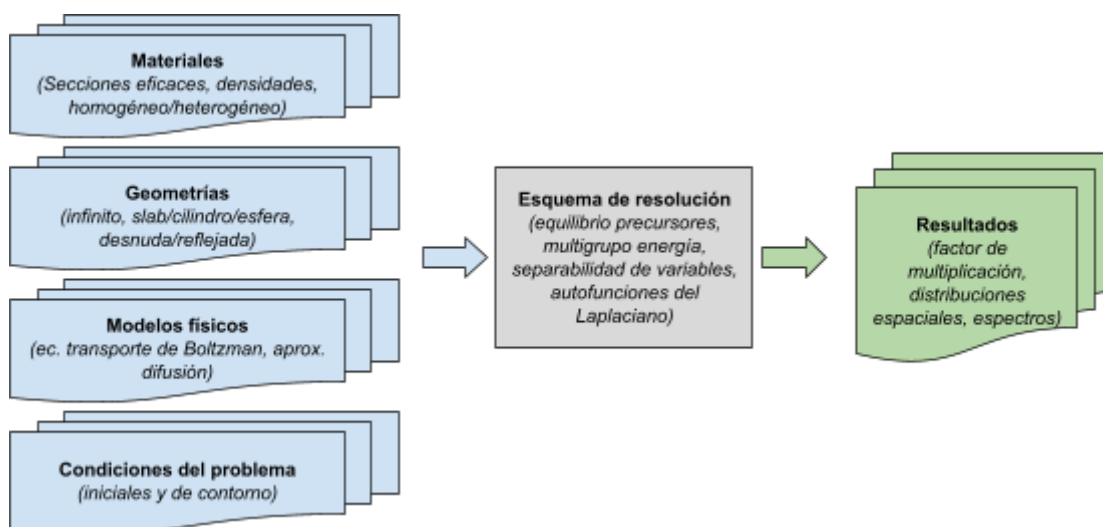


Inducción a las herramientas y conceptos

1 Introducción

Cuando se resuelve un problema analíticamente, normalmente se sigue un esquema de trabajo como el siguiente:



Las soluciones analíticas poseen numerosas ventajas, pero su implementación se puede volver muy complicada cuando las geometrías no son simples, cuando se quiere aumentar el detalle energético, o cuando el sistema posee heterogeneidades. En estas circunstancias es cuando se recurre a la resolución del problema mediante el uso de computadoras. En particular, se puede observar que:

- Los materiales ya se encuentran en bases de datos, o sea digitalizadas.
- Las condiciones de contorno en general se pueden expresar bien en el lenguaje de los códigos de cálculo:
 - Flujo entrante nulo en la frontera del dominio, o flujo nulo en la frontera extrapolada
 - Condición especular donde haya simetría
- Las ecuaciones integro-diferenciales en general no se pueden resolver algebraicamente en los programas, salvo casos relativamente simples.
- Las geometrías en general tienen que ser descritas como un conjunto de formas simples, llamadas mallas. Una excepción a esto son las descripciones [NURBS](#) de las geometrías, en donde las mismas son expresadas como funciones básicas algebraicas, las cuales sufren diversas transformaciones espaciales.

Por lo tanto, a continuación se va a realizar una introducción a algunas de las soluciones implementadas para resolver estos últimos dos inconvenientes.

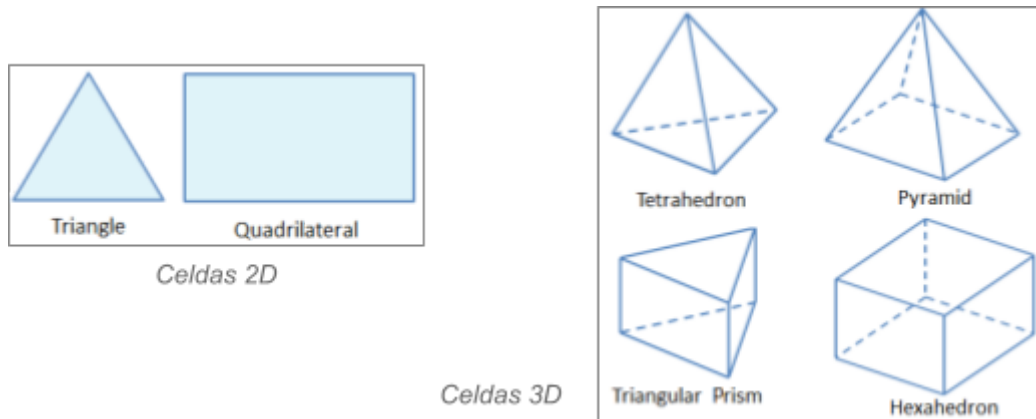
Mallado de la geometría

La discretización del dominio donde se quiere resolver el problema se realiza normalmente mediante lo que se denomina mallado.

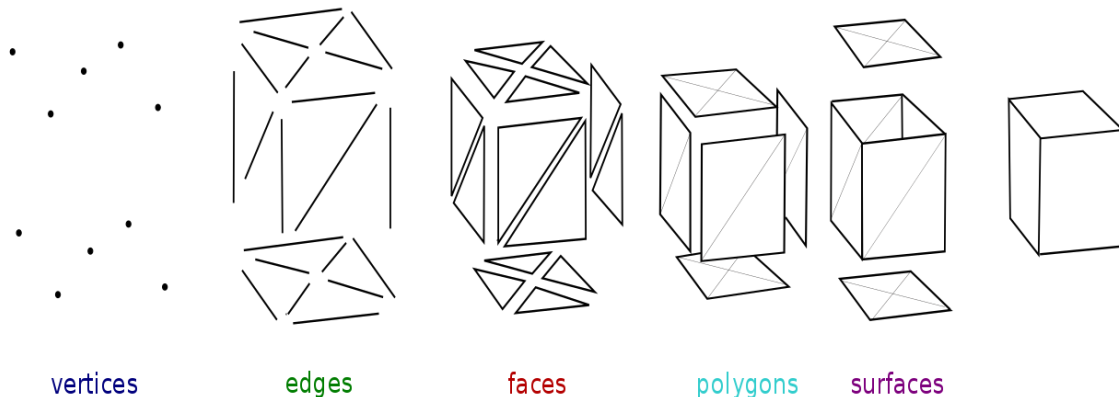
Por fortuna, el mallado no es una herramienta exclusiva de la resolución numérica de problemas de reactores, sino que es usado también para resolver problemas físicos en general, y además es muy utilizado en lo que se denomina “[computación gráfica](#)” (renderizado de texturas para imágenes digitales, videos, animaciones, videojuegos, realidad aumentada, etc.), y como lenguaje de comunicación con las impresoras 3D.

Características de las mallas

Las mallas están formadas por un número finito de unidades básicas denominadas celdas, las cuales suelen ser las mostradas en la [siguiente figura](#):

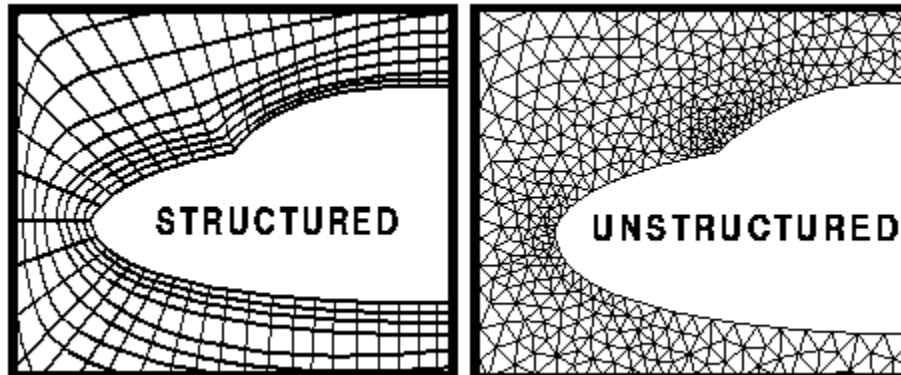


Es importante remarcar que según la cantidad de dimensiones que se esté trabajando, depende la cantidad de propiedades que son necesarias utilizar para [definir a las celdas](#):



Por otro lado, otra característica importante a definir en las mallas es su calificación en:

- Estructurada: en este caso el mallado posee una distribución regular, en la cual resulta más sencilla la implementación de la resolución numérica de las ecuaciones a trabajar, pero puede ser más limitada o ineficiente la descripción del dominio.
- No estructuradas: en este caso el tamaño y disposición de las celdas no sigue una distribución regular, sino que se adapta a los lugares que requieren mayor detallado (ya sea por la forma del dominio o por las características de las soluciones del sistema)

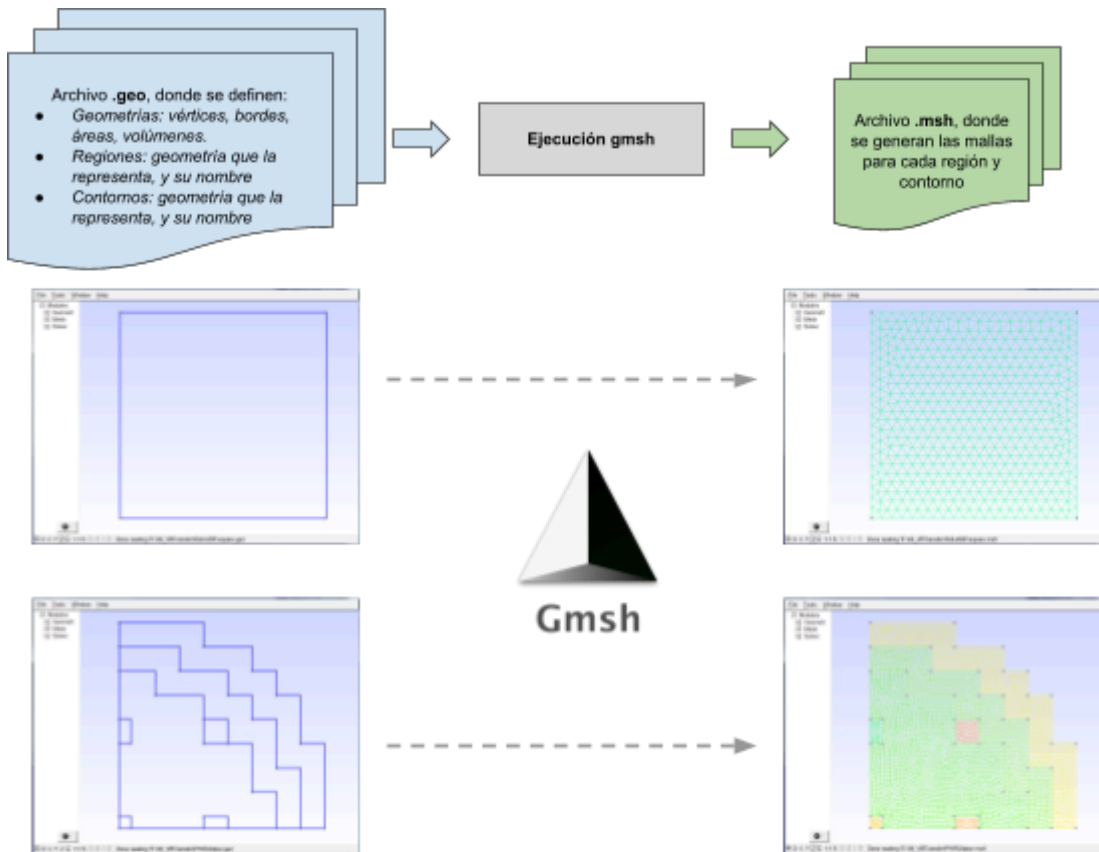


Generación de las mallas

Existen diversas formas de generar una malla. Lo más común es [utilizar la asistencia de algún programa para su generación](#), partiendo de dibujos del dominio realizados con algún programa tipo CAD (*Computer-Aided Drawing*), y luego generando la malla en el mismo programa o exportando el dibujo a otro programa específico de generación de mallas.

Sin embargo, dado que en esta práctica se trabaja con geometrías simples, entonces las mallas se van a generar por líneas de comandos, permitiendo cambiar fácilmente sus características, para realizar estudios paramétricos.

Por ello, las mallas se van a generar con la herramienta **gmsh** (<https://gmsh.info/>) a partir de un archivo de definición geométrica **.geo**, el cual es el archivo de entrada del programa gmsh.



En el archivo ASCII .msh de salida se guarda la información de la malla creada con gmsh. Tiene información de los nodos y los elementos que surgen de la discretización. Además se guarda la información de las superficies exteriores y regiones (*physical entities*).

Resolución numérica de ecuaciones diferenciales

Para resolver las ecuaciones diferenciales la idea que se usa es la de obtener un sistema matricial que represente al problema, partiendo de las ecuaciones continuas de un espacio continuo. A lo largo de la materia ya se vieron dos técnicas para obtener sistemas algebraicos a partir de ecuaciones diferenciales: una fue la utilización de la transformada de Laplace, y la otra fue la obtención de un esquema multigrupo de energías.

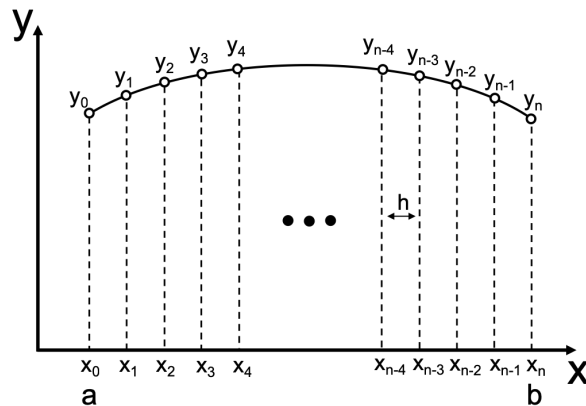
Pero para sistemas más generales hay diversas técnicas para obtener esta transformación de espacios matemáticos, las cuales aquí solamente se va a realizar una introducción extremadamente breve, ya que cada una de las técnicas son suficientemente complejas como para poder ser dadas en materias completas de grado o post-grado. Varias de las imágenes y conceptos que se dan en esta sección fueron sacadas de [algunas](#) páginas de [internet](#), las cuales se recomienda su lectura si se quiere continuar con la introducción al tema. En particular, se recomienda leer [la monografía](#) del autor del FeenoX, donde explica estos temas con más detalle.

En general, las matrices que se obtienen vinculan el valor de la función que se está queriendo resolver, con los valores de la misma función en los puntos de su entorno, lo cual produce que la matriz [sea dispersa](#). Luego la resolución del problema se convierte en la obtención de los autovalores de la matriz.

Al utilizar un método numérico, lo que se obtiene es una aproximación de la solución u del problema, y a esta aproximación se la puede denominar por ejemplo u_h . El hecho de obtener una solución aproximada tiene como consecuencia que se obtenga un error en la estimación de la solución, el cual se denomina *error de discretización*: $e = \|u - u_h\|$. Lo que se espera es que este error disminuya al refinar la discretización, pero debido a errores de truncamiento puede pasar que este error tenga una cota de la cual no se puede bajar el mismo.

Diferencias finitas

Una de las estrategias que se usa para obtener el sistema matricial es el de discretizar los operadores, mediante la técnica denominada **diferencias finitas**. La idea detrás de esta técnica es la de aprovechar que el dominio se encuentra discretizado, para calcular los operadores como diferencias entre los valores de las funciones en los puntos del dominio:



Por ejemplo, si se trabaja en un sistema unidimensional y si se denomina como h a la distancia entre dos puntos del dominio discretizado, [se tiene](#):

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

$$f''(x) \approx \frac{\frac{f(x+h)-f(x)}{h} - \frac{f(x)-f(x-h)}{h}}{h} = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

Está técnica resulta más eficiente de implementar cuando se trabaja con mallas regulares, ya que de esta manera h pasa a ser una constante.

Como se puede apreciar, finalmente el valor de la función en un punto pasa a depender de los valores de la función en sus puntos vecinos, y por lo tanto esto se puede expresar como una relación matricial, que como se dijo anteriormente, va a ser una matriz llena de ceros, excepto en unos pocos lugares..

Volúmenes finitos

A [modo de ejemplo](#), se puede suponer que los sistemas de ecuaciones que se resuelven con este método son del estilo:

$$\frac{\partial u}{\partial t} + \nabla \cdot \Gamma = F \text{ in } \Omega$$

Donde en nuestro caso, u es la distribución del flujo de neutrones, el gradiente de esa ecuación suele ser en realidad el operador de difusión, F es la fuente de neutrones, y Ω es el dominio (o sea el núcleo y el reflector del reactor).

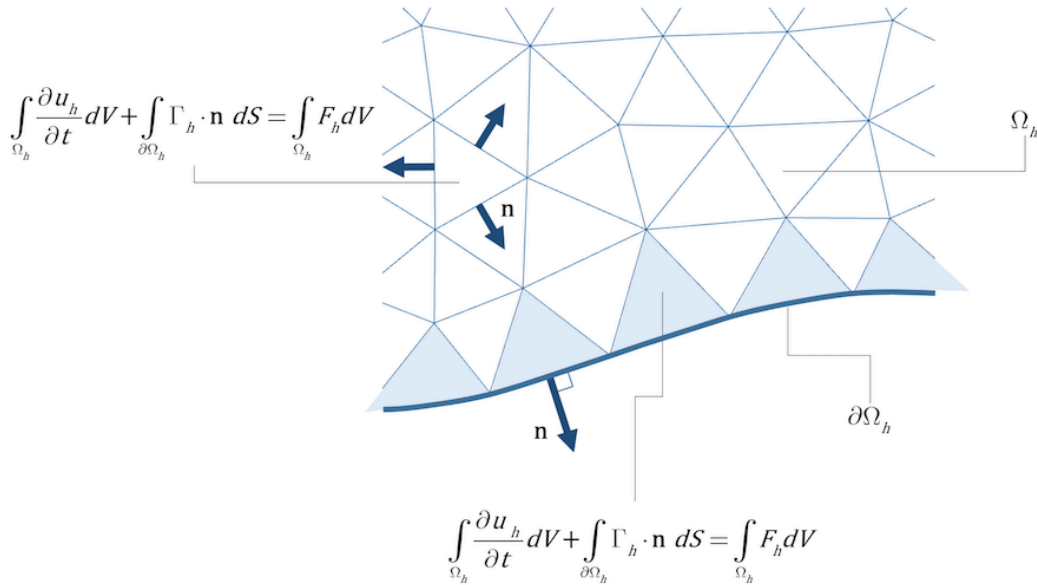
Si a la ecuación anterior se la pesa con una función de peso φ , luego se la integra en el dominio, y finalmente se separa el gradiente por partes, se obtiene:

$$\int_{\Omega} \frac{\partial u}{\partial t} \varphi dV - \int_{\Omega} \Gamma \cdot \nabla \varphi dV + \int_{\partial\Omega} \Gamma \cdot \mathbf{n} \varphi dS = \int_{\Omega} F \varphi dV$$

La cual es el tipo de ecuación que es la base para la implementación de los métodos de **volúmenes finitos** y de **elementos finitos**.

En el caso de los volúmenes finitos, la integral se realiza en realidad en cada celda (en vez de en todo el dominio), y el peso es igual a uno. De esta forma se pierde la información de lo que pasa dentro de la celda, obteniéndose simplemente el valor total (o promedio) de la función en la celda.

Por otro lado, cada celda (o volumen) se vincula con las celdas vecinas mediante el flujo por las fronteras entre las celdas, o posee parte de su frontera definida como condición de contorno. Normalmente en el flujo es donde se realiza una aproximación, estimando este flujo como una diferencia entre los valores promedio de las celdas.



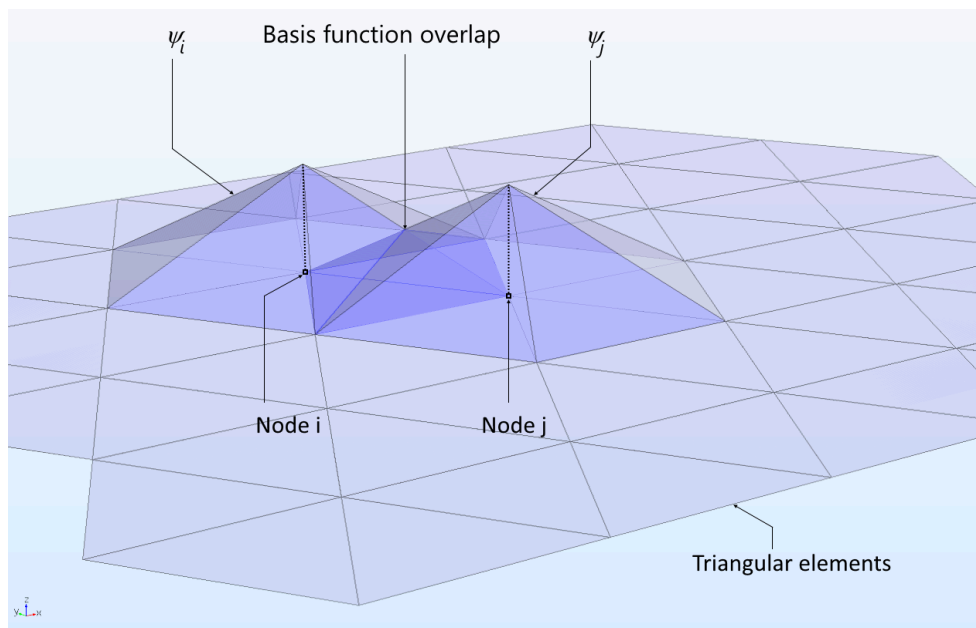
Como se puede apreciar, el nombre de la técnica viene del hecho de subdividir al dominio en un conjunto de volúmenes, y calcular las propiedades promedio de cada uno de esos volúmenes.

Elementos finitos

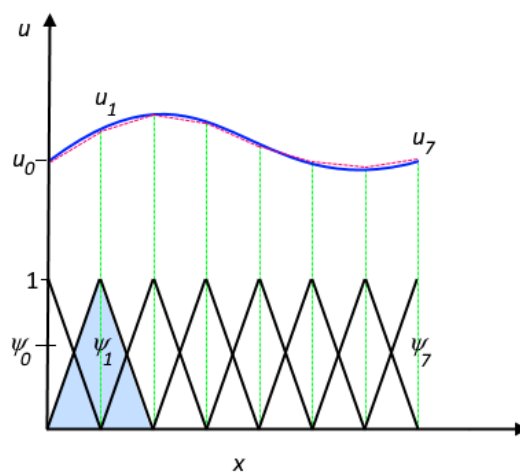
Esta técnica se basa en proponer que la solución aproximada se puede expresar como una sumatoria finita de unas funciones ψ_i (que pasan a ser los elementos finitos), multiplicadas por los pesos correspondientes:

$$u_h = \sum_i u_i \psi_i$$

Estas funciones arbitrarias normalmente suelen ser funciones simples (como rectas o polinomios) que vinculan a un nodo de la malla con sus vecinos. Normalmente los nodos suelen ser los vértices de las celdas de la malla:



En la [siguiente figura](#) se muestra cómo la solución puede ser aproximada mediante la sumatoria expresada anteriormente. Además, se creó un gráfico en la [página desmos](#) para probar esta aproximación manualmente.



Dado que estos elementos finitos vinculan solamente a un nodo con sus primeros vecinos, entonces nuevamente se obtiene que la solución se expresa como una matriz dispersa.

Aplicaciones más comunes de estas técnicas:

Las aplicaciones más comunes de cada una de estas técnicas son:

- **Elementos finitos:** Análisis estructurales, transferencia de calor, ingeniería química, electromagnetismo, multi-física, CFD (*Computational Fluid Dynamics*).
- **Diferencias finitas:** Cálculos meteorológicos, astrofísica, sismología, realismo físico en computación gráfica, efectos especiales.
- **Volúmenes finitos:** CFD, transferencia de calor, ingeniería química.

FeenoX como herramienta para el cálculo de difusión de neutrones

En la siguiente sección se presenta una breve guía de uso de [FeenoX](#) como herramienta para el cálculo de difusión de neutrones.

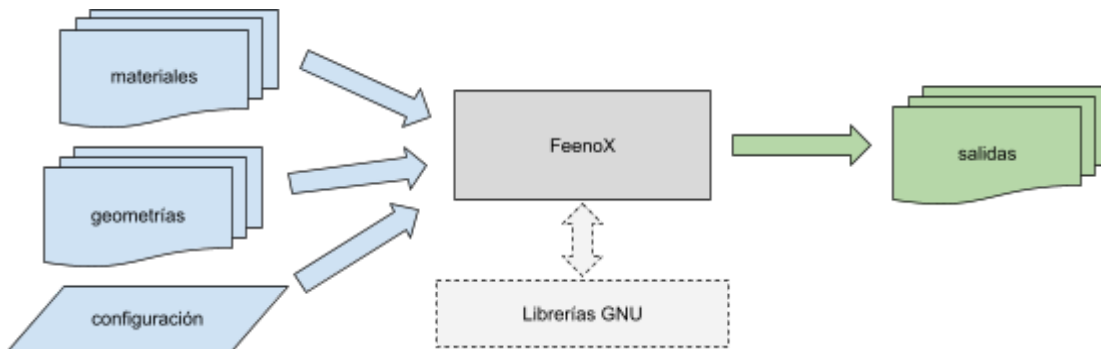
La información presentada fue recopilada de los trabajos disponibles en la página de [FeenoX](#) sumado a algunos aspectos técnicos y buenas prácticas de uso general en cálculo de reactores.

Dentro de las opciones de resolución de FeenoX se encuentra disponible la posibilidad de llevar a cabo cálculos neutrónicos a nivel de núcleo.

Esta herramienta resuelve la ecuación de transporte multigrupo en estado estacionario utilizando la aproximación de difusión o método de ordenadas discretas. Para ello se puede emplear un esquema numérico de elementos finitos sobre una malla que sea estructurada, o no.

Esquema de funcionamiento

FeenoX lee un conjunto de archivos de entrada dónde se detallan los parámetros necesarios para definir adecuadamente el problema a resolver. Luego procesa esta información y se generan las matrices P, A y F para resolver el problema de un RCAk y obtener el factor de multiplicación y la distribución de flujo.



Como en todo proceso de cálculo, es necesario especificar los datos de entrada adecuados para resolver el problema en cuestión y las instrucciones que definen los parámetros de salida para el posterior análisis de resultados. En este caso particular es necesario definir:

- A. **Geometría:** Se especifican las dimensiones del “reactor” (slab, cilindro, esfera, paralelepípedo, etc), regiones que lo componen (región combustible, moderador, reflector, barras de control, etc), la discretización espacial (malla) y condiciones de contorno (vacío, reflejado, etc).
- B. **Materiales:** Para cada región definida en la geometría, se especifican las secciones eficaces macroscópicas para los diferentes grupos de energía, como así también los coeficientes de difusión y el espectro de fisión.
- C. **Esquema de cálculo:** Método de resolución de la ecuación de transporte; aproximación por difusión o método de ordenadas discretas, empleando elementos finitos.
- D. **Tipo de cálculo:** Se define qué parámetros se desean obtener; por ejemplo Keff, flujos neutrónicos, espectros espaciales, distribución de potencia, factor de pico, etc. FeenoX por defecto calcula el Keff y flujo (autovalor y autovectores).
- E. **Salida:** Se especifican los tipos de resultados a procesar; por ejemplo, keff, reactividad, Keff vs algún parámetro (análisis de sensibilidad), distribución espacial de flujos a diferentes energías, posición de máximo de flujo, factor de pico en función del quemado, distribución espacial de secciones eficaces, etc.

- F. **Expresiones algebraicas (opcional):** Se pueden definir constantes, vectores, matrices y funciones. Además se pueden usar funciones básicas como log, exp, sen, cos, etc o funciones predefinidas como integración, derivación, búsqueda del máximo, etc. Este atributo permite hacer operaciones simples, previas a la resolución de la ecuación de transporte y también para manipular los resultados de salida.

Por ejemplo, se desea obtener el Keff en función del quemado a diferentes estados de operación, entonces podemos definir las secciones eficaces como funciones del quemado y temperatura, $\Sigma_x(Bu, T)$, mediante su expresión funcional o a través de una tabla.

Estos datos de entrada se pueden especificar en uno o varios archivos de texto mediante las palabras claves que los definen, las cuales se encuentran especificadas en el manual de [FeenoX](#).

Por defecto los parámetros de salida (E) se imprimen en la terminal, pero pueden ser direccionados a un archivo de texto o archivo de post-procesamiento (.vtk)

¿Cómo se escribe un archivo de entrada?

Los archivos de entrada (de ahora en adelante *input*) se escriben empleando palabras claves autodescriptivas, teniendo en mente la filosofía de “problemas simples deberían tener inputs simples”.

Para comenzar, siempre es conveniente crear un input con un nombre adecuado al problema que estamos resolviendo y si es posible una extensión (aunque no es necesaria) para identificar qué tipo de herramienta estamos usando. Por ejemplo, [Slab_reflejado.fee](#):

Datos Generales

- 1) Para ser ordenados y claros, es recomendable comenzar a escribir un *input* con líneas de comentarios. Aquí es donde vamos a poner una breve descripción del problema que estamos analizando y alguna otra información que consideremos importante. Por ejemplo:

```
# Descriptive Title: Two-dimensional LWR Problem,
#                   also 2D IAEA Benchmark Problem
#
# Reduction of Source Situation
#       1. Two-group diffusion theory
#       2. Two-dimensional (x,y)-geometry
#
```

- 2) Luego de los comentarios se especifica el tipo de problema a resolver

```
PROBLEM neutron_diffusion 3D GROUPS 2
```

```
PROBLEM neutron_diffusion 2D GROUPS 5
```

```
PROBLEM neutron_sn DIM 1 GROUPS 1 SN 6
```

- 3) A continuación, datos iniciales del problema como definiciones de constantes, funciones, vectores, etc.

```
DEFAULT_ARGUMENT_VALUE 1 20
DEFAULT_ARGUMENT_VALUE 2 160
DEFAULT_ARGUMENT_VALUE 3 260
#
FUNCTION f(x) FILE_PATH f.dat INTERPOLATION akima
#
```

```
MATRIX PPF ROWS 3 COL 4
ngroups=3
ncells=40
deltax=1/ncells
VECTOR FLUX SIZE ngroups*ncells
```

Geometría

En FeenoX la geometría y discretización del problema a resolver se especifican mediante un archivo **.msh**, ya sea generado mediante el **gmsh**, o mediante alguno de los otros generadores de mallas.

El archivo **.msh** se ingresa en el input de FeenoX mediante la tarjeta **READ_MESH**.

Si ya tenemos el archivo de malla **.msh** generado con **gmsh** en forma externa lo podemos ingresar de la siguiente forma:

```
# Malla no estructura definida en un cuarto de cilindro

DEFAULT_ARGUMENT_VALUE 1 quarter # quarter o eighth
READ_MESH iaea-3dpwr-$1.msh
```

```
# leer la malla de este archivo
READ_MESH reed.msh
```

Materiales

Las secciones eficaces macrocópicas se ingresan a las diferentes regiones definidas e identificadas en el archivo de geometría **.msh** mediante la tarjeta **MATERIAL**.

En el siguiente ejemplo podemos notar que el reactor está formado por 4 regiones (*physical entities*), identificadas como **fuel1**, **fuel2**, **fuel2rod** y **reflector**.

```
# define materials and cross sections according to the two-group constants
# each material corresponds to a physical entity in the geometry file
#
Bg2 = 0.8e-4 # axial geometric buckling in the z direction
MATERIAL fuel1 {
  D1=1.5      Sigma_a1=0.010+D1(x,y)*Bg2      Sigma_s1.2=0.02
  D2=0.4      Sigma_a2=0.080+D2(x,y)*Bg2      nuSigma_f2=0.135 }#eSigmaF_2 nuSigmaF_2(x,y) }

MATERIAL fuel2 {
  D1=1.5      Sigma_a1=0.010+D1(x,y)*Bg2      Sigma_s1.2=0.02
  D2=0.4      Sigma_a2=0.085+D2(x,y)*Bg2      nuSigma_f2=0.135 }#eSigmaF_2 nuSigmaF_2(x,y) }

MATERIAL fuel2rod {
  D1=1.5      Sigma_a1=0.010+D1(x,y)*Bg2      Sigma_s1.2=0.02
  D2=0.4      Sigma_a2=0.130+D2(x,y)*Bg2      nuSigma_f2=0.135 }#eSigmaF_2 nuSigmaF_2(x,y) }

MATERIAL reflector {
  D1=2.0      Sigma_a1=0.000+D1(x,y)*Bg2      Sigma_s1.2=0.04
  D2=0.3      Sigma_a2=0.010+D2(x,y)*Bg2 }
```

Una vez que el archivo de geometría se procesa, las secciones eficaces definidas son tratadas como funciones de las variables espaciales (en el ejemplo anterior x e y) para que se puedan evaluar, integrar, etc.

También se pueden definir mediante tablas como se muestra a continuación.

```
# Distribucion de quemado q(x,y), si beq=0 entonces indica beginning of life y q(x,y)=0
# Hacemos q(x,y)=0 si y<0, y>h o |x|>r, para que al escribir el quemado en funcion de x e y sepamos
```

```
# bien donde termina

q(x,y):=beq*(1000+7000*sin(pi*y/h)*(1-(x/r)^6))*(y>0 & y<h & abs(x) < r)

# XS en funcion del quemado
FUNCTION dif1(b)      FILE xs.dat COLUMNS 1 2
FUNCTION dif2(b)      FILE xs.dat COLUMNS 1 3
FUNCTION sigmaa1(b)    FILE xs.dat COLUMNS 1 4
FUNCTION sigma12(b)    FILE xs.dat COLUMNS 1 5
FUNCTION sigma21(b)    FILE xs.dat COLUMNS 1 6
FUNCTION sigmaa2(b)    FILE xs.dat COLUMNS 1 7
FUNCTION nusigf1(b)    FILE xs.dat COLUMNS 1 8
FUNCTION nusigf2(b)    FILE xs.dat COLUMNS 1 9

MATERIAL fuel {
D_1      dif1(q(x,y))
D_2      dif2(q(x,y))
SigmaT_1  sigmaa1(q(x,y))+sigma12(q(x,y))
SigmaS_1.2 sigma12(q(x,y))
SigmaS_2.1 sigma21(q(x,y))
SigmaT_2  sigmaa2(q(x,y))+sigma21(q(x,y))
nuSigmaF_1 nusigf1(q(x,y))
nuSigmaF_2 nusigf2(q(x,y))
}
MATERIAL refl {
D_1      1.40
D_2      0.95
SigmaT_1  1.51e-4+1.23e-2
SigmaS_1->2 1.23e-2
SigmaS_2->1 6.89e-5
SigmaT_2  1.06e-3+6.89e-5
}
}
```

Condiciones de borde

Las condiciones de borde se especifican en las superficies exteriores, identificadas mediante un nombre en el archivo de malla **.msh**. Se pueden especificar; condiciones de contorno de flujo nulo en el borde, vacío, especular y reflectivas dadas mediante un albedo. Por ejemplo:

- a) *Slab con flujo nulo en los bordes*: En el archivo **.msh** se identificaron los extremos como *left* y *right*.

```
# condiciones de contorno
BC left mirror
BC right vacuum
```

- b) *Cuarto de esfera con condición de borde especular y vacío*: En el archivo **.msh** se identificaron a las superficies del borde como *internal* (parte plana interior) y *external* (parte curva exterior).

```
# set boundary conditions
BC internal mirror
BC external vacuum
```

- c) *Cuarto de núcleo de un PWR con condiciones de borde especular y reflectiva*: En el archivo **.msh** se identificaron a las superficies del borde como *mirror* (superficie izquierda e inferior) y *external* (parte escalonada).

```
# condiciones de contorno
BC external vacuum 0.4692
BC mirror mirror
```

Si no se ingresa ninguna condición de borde se asume condición de borde de **flujo nulo** en las superficies exteriores.

Obtención de la solución

Hasta este punto el problema se encuentra bien definido; especificamos geometría, materiales y condiciones de borde.

Ahora debemos resolver el problema neutrónico (problema de autovalores) y para ello empleamos la siguiente tarjeta:

```
# Resolvemos el problema
SOLVE PROBLEM
```

Esta tarjeta calcula el factor de multiplicación y el flujo en función de la energía y el espacio. Los valores son guardados en las variables **Keff** y **phi1** (grupo 1), **phi2** (grupo 2), **phi3** (grupo 3), etc.

Resultados de Salida (Output)

Finalmente se debe especificar qué se quiere imprimir como salida del problema.

a) Mediante la tarjeta **PRINT** se puede imprimir en la salida estándar (terminal).

```
# Imprimimos flujo
# Imprimimos Keff (resultado de cálculo) vs Kdiff (resultado teorico)
kdiff = nuSigmaF(0) / ((SigmaT(0)-SigmaS(0))+D(0)*(pi/a)^2)

PRINT %.5f "keff = " keff
```

b) Escribimos los resultados de flujos o distribución de secciones eficaces en un archivo de pos procesamiento (.vtk o .msh) con la tarjeta **WRITE_RESULTS** (escribe los resultados en un archivo de post procesamiento en formato .msh o .vtk).

```
PRINT TEXT "\# --- $1 ---"
PRINT TEXT "\#   keff       = " %.8f keff SEP " " TEXT "(" %+1f 1e5*(1-1/keff) TEXT "pcm )"
PRINT TEXT "\#   nodes      = " %.0f nodes
PRINT TEXT "\#   elements   = " %.0f elements
PRINT TEXT "\#   CPU usage: "
PRINT TEXT "\#   init       = " %.3f time_cpu_ini      TEXT "seconds"
PRINT TEXT "\#   build      = " %.3f time_cpu_build   TEXT "seconds"
PRINT TEXT "\#   solve      = " %.3f time_cpu_solve   TEXT "seconds"
PRINT TEXT "\#   total      = " %.3f time_cpu_total   TEXT "seconds"
PRINT TEXT "\#   Memory: "
PRINT TEXT "\#   used       = " %.0f 1e-6*memory_usage_global TEXT "Mb"
PRINT TEXT "\#   available = " %.0f 1e-6*available_memory   TEXT "Mb"
# files for gmsh and vtk post-processing
PRINT_FUNCTION phil
WRITE_RESULTS FORMAT vtk
```

c) Imprimimos en pantalla y en un archivo de texto funciones definidas en el problema

```
# Comparar Keff y phi (calculado) con valores teóricos
# PRINT %g TEXT "a = " a TEXT "b = " b TEXT "n = " n TEXT "lc = " lc
PRINT "keff analytical = " %.8f k
PRINT "keff numerical   = " %.8f keff
PRINT "diference        = " %e keff-k
PRINT_FUNCTION flux MIN 0 MAX a STEP a/1000 FILE two-zone-analytical.dat
PRINT_FUNCTION phil phil(x)-flux(x)          FILE two-zone-numerical.dat
```

Otros aspectos generales a tener en cuenta

Al momento de definir una constante, variable o función es importante dejar un espacio antes y después del signo igual.

```
# include some files that we need for the grid
a = 50
A = 1.57
Rho = (Keff-1)/Keff
```

```
Flux := A*sin(Pi*x/a)
```

Para incluir un archivo al input principal se usa la tarjeta **INCLUDE**:

```
# include some files that we need for the grid
INCLUDE 2dpwr_ests.fee
INCLUDE 2dpwr_materiales.fee
```

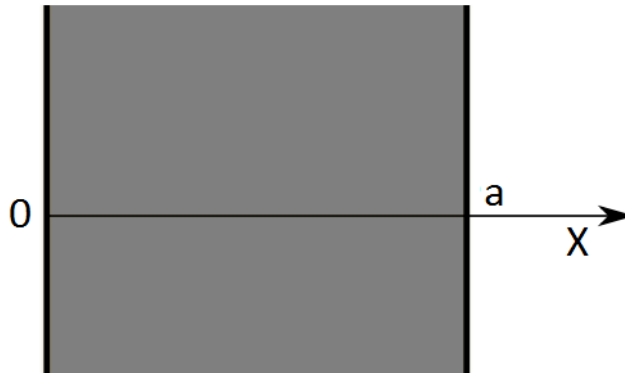
Algunos ejemplos

A continuación se muestran algunos ejemplos para comenzar a utilizar el FeenoX. Para ver más ejemplos se recomienda ver los ejemplos en las páginas respectivas de los predecesores del Feenox (el Milonga y el Wasora):

- Milonga: <https://github.com/seamplex/milonga/tree/master/examples>
- Wasora: <https://github.com/seamplex/wasora/tree/master/examples>
- FeenoX: <https://www.seamplex.com/feenox/doc/#examples>

Slab Desnudo

Se presenta la resolución de un problema tipo slab desnudo unidimensional a un grupo de energía de ancho a . La superficie izquierda está ubicada en $x=0$ y la derecha en $x=a$.



Se busca calcular el k_{eff} , flujo y factor de pico para compararlos con resultados teóricos.

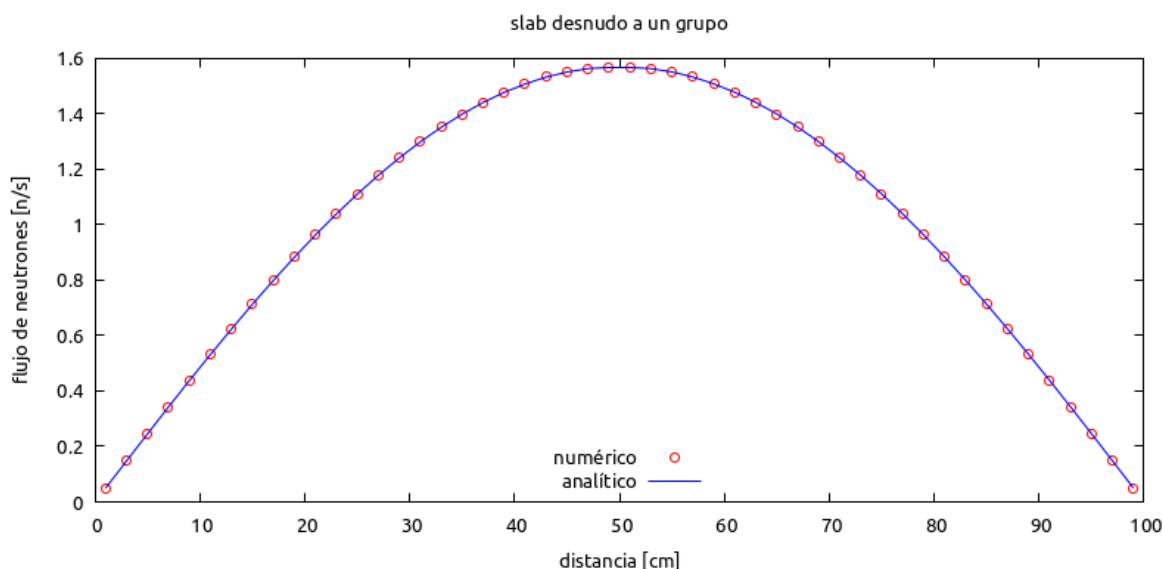
A continuación se muestran los resultados en base a los archivos que están en el directorio [01_slab-desnudo](#). Para ver el ejemplo, simplemente hay que ejecutar el script *correEjemplo.sh*.

Una vez ejecutado, se va a ver en la pantalla la siguiente salida:

```
fr@fr-VirtualBox: ~/Escritorio/intercambio/TP4/00_ejemplos/01_slab-desnudo$ ./correEjemplo.sh
Generando la malla ...
Info : Running 'gmsh -1 -algo auto slab.geo -o out.slabs.msh' [Gmsh 4.8.4, 1 node, max. 1 thread]
Info : Started on Sat Apr 13 13:44:39 2024
Info : Reading 'slab.geo'...
Info : Done reading 'slab.geo'
Info : Meshing 1D...
Info : Meshing curve 1 (Line)
Info : Done meshing 1D (Wall 0.000250384s, CPU 0.000161s)
Info : 51 nodes 52 elements
Info : Writing 'out.slabs.msh'...
Info : Done writing 'out.slabs.msh'
Info : Stopped on Sat Apr 13 13:44:39 2024 (From start: Wall 0.00619942s, CPU 0.055427s)
Resolviendo el reactor ...
# keff = 1.00012591
# keff_t = 1.00012912
# keff error = -3.22e-06
# fp = 1.57136037
# fp_t = 1.57079633
# fp error = +3.59e-04
Graficando los resultados ...
Terminado
fr@fr-VirtualBox:~/Escritorio/intercambio/TP4/00_ejemplos/01_slab-desnudo$
```

Donde las líneas que comienzan con “Info” son las salidas de la ejecución del Gmsh, mientras que las restantes son el resultado de FeenoX.

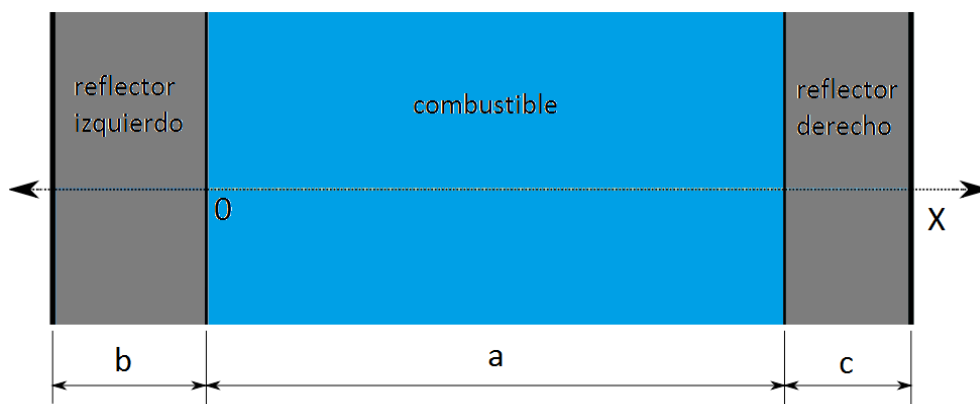
Además, entre otros archivos, genera el siguiente gráfico comparativo entre el cálculo analítico y el numérico. En este gráfico se puede apreciar que existe una buena concordancia entre ambos cálculos.



Este tipo de validación es uno de los tantos que posee el FeenoX, entre los que además hay varios *Benchmark* de complejidad elevada.

Slab Reflejado

Se presenta la resolución de un problema tipo slab reflejado unidimensional a dos grupos de energía de ancho a para el combustible, b para el reflector izquierdo y c para el reflector derecho. La superficie izquierda está ubicada en $x=-b$ y la derecha en $x=a+c$.

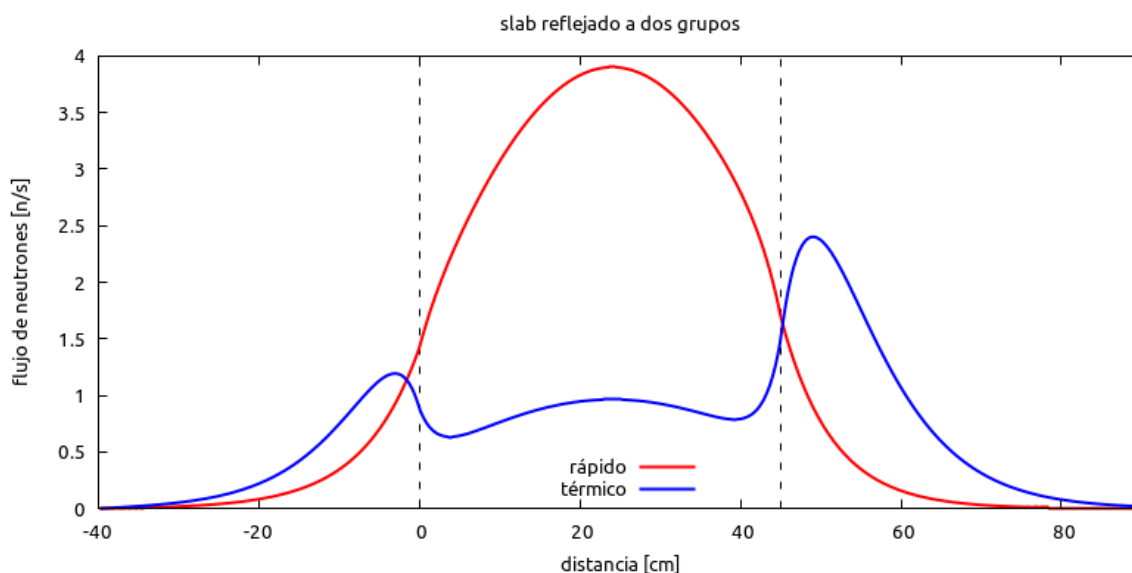


1.1

Se busca calcular el k_{eff} , flujo a dos grupos y el espectro S_1 .

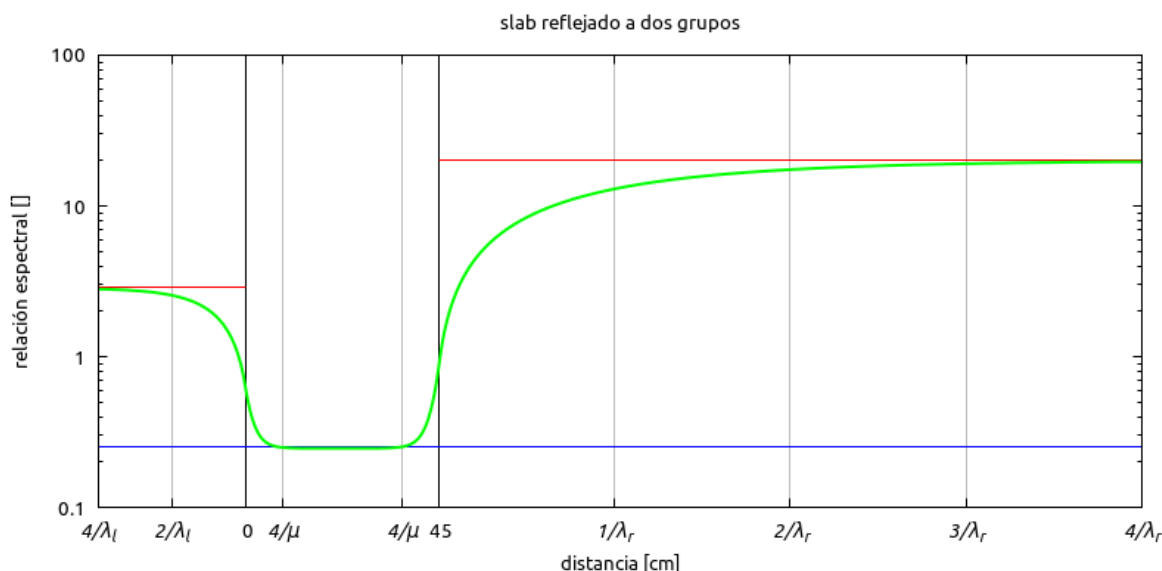
A continuación se muestran los resultados en base a los archivos que están en el directorio [02_slab-reflejado](#). Al igual que el ejemplo anterior, este puede ser reproducido ejecutando el script que se encuentra dentro.

Lo interesante de este ejemplo es que posee una distribución asimétrica de reflectores, tanto en su espesor como en su composición. Esta asimetría puede apreciarse en la siguiente figura, especialmente en lo que respecta al flujo térmico.



Otro aspecto interesante que se obtiene en este ejemplo, es la visualización de las zonas donde no vale la separabilidad de variables, lo cual se ve reflejado en el gradiente de la relación espectral

mostrado en la siguiente figura.

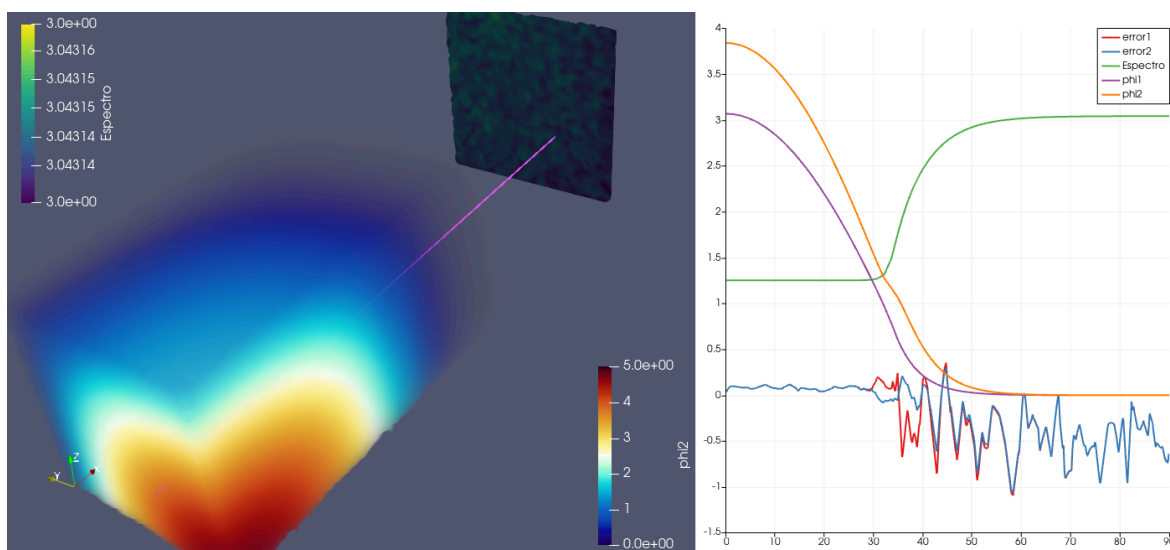


Donde:

- μ es el segundo autovalor de la matriz energía del combustible,
- $\lambda_l = \lambda_{3l} + \lambda_{4l}$, siendo λ_{3l} el primer autovalor de la matriz energía del reflector de la izquierda, y λ_{4l} su segundo autovalor,
- λ_r el equivalente a λ_l pero para el reflector de la derecha.

Visualización de resultados

En caso de trabajar con geometrías en 1D o en 2D, el uso del GNUPlot seguramente resultará suficiente para visualizar los resultados. Sin embargo, cuando se trabaja con geometrías 3D, es muy recomendable la utilización del programa [ParaView](#), ya que posee múltiples herramientas para trabajar con este tipo de información, como el mostrado en la siguiente figura:

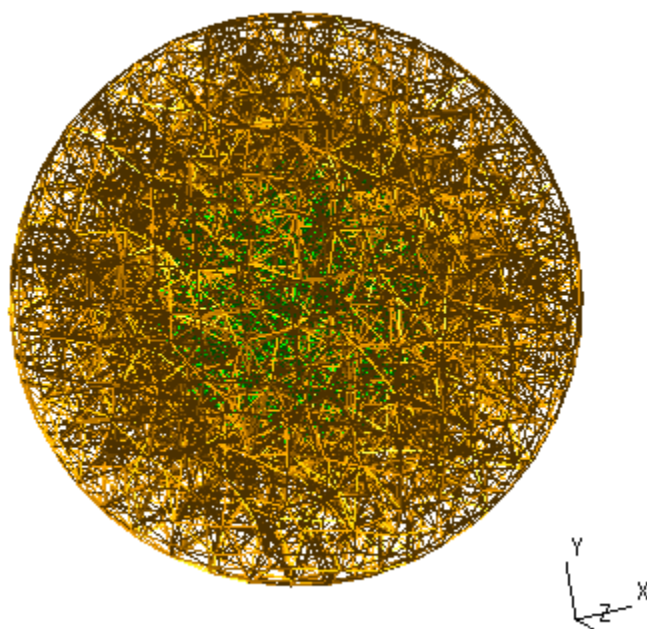


Esta figura es el resultado del modelado de un núcleo con forma de cubo, el cual está reflejado en la dirección x. A la izquierda de la figura se muestra una representación volumétrica del flujo térmico, además de un corte del espectro lejos del núcleo. Mientras que a la derecha se muestran diversas variables a lo largo de una línea paralela al eje x.

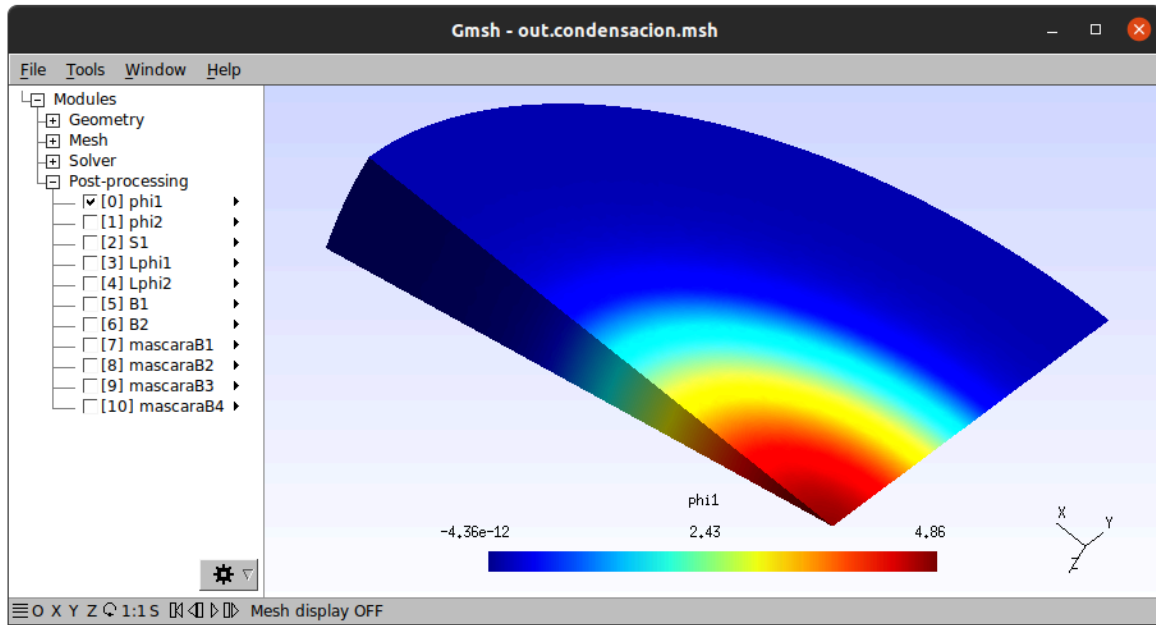
Sin embargo, se mostrará aquí brevemente que el mismo Gmsh también posee capacidad de mostrar los resultados de las simulaciones, y para ello se utilizarán los archivos que están dentro de la carpeta [03 esfera](#).

En este ejemplo se simula una esfera reflejada. Este tipo de geometría posee la particularidad de ser en realidad un problema unidimensional, ya que posee simetría de revolución. Esta simetría hace que no sea necesario simular la esfera entera, sino que se puede simular simplemente una fracción de la misma, con el objetivo de optimizar los recursos de cálculo.

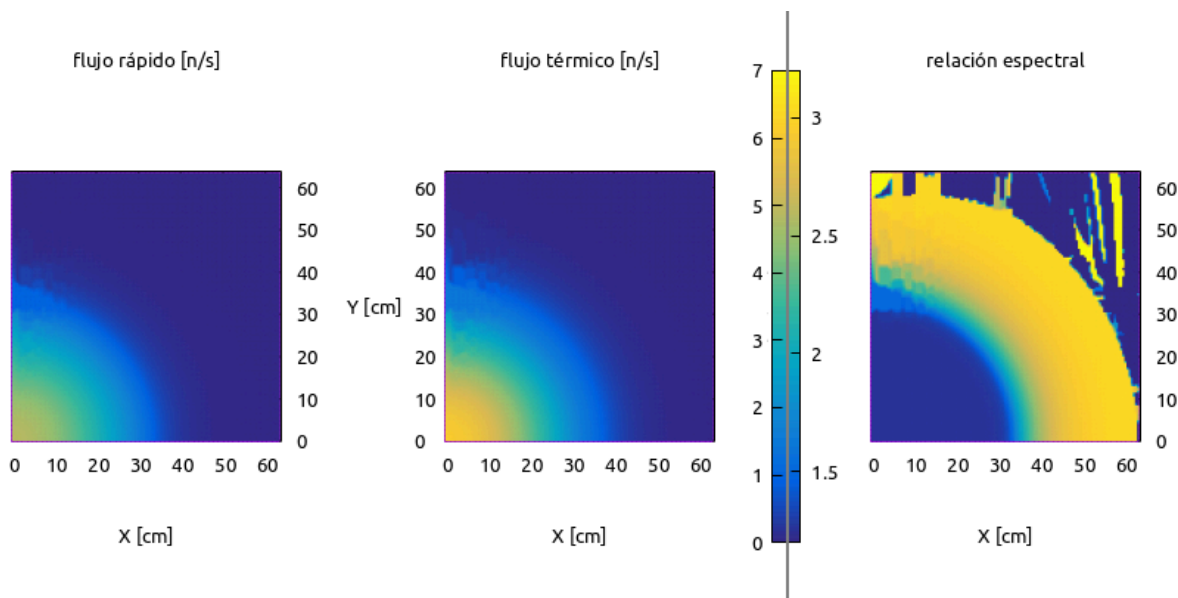
En la siguiente figura se muestra un mallado relativamente grueso del dominio del problema. El núcleo se muestra con una coloración verde, y el reflector con una coloración naranja. La animación fue realizada con el Gmsh, y se puede apreciar que puede ser difícil de interpretar incluso algo tan simple como el mallado.



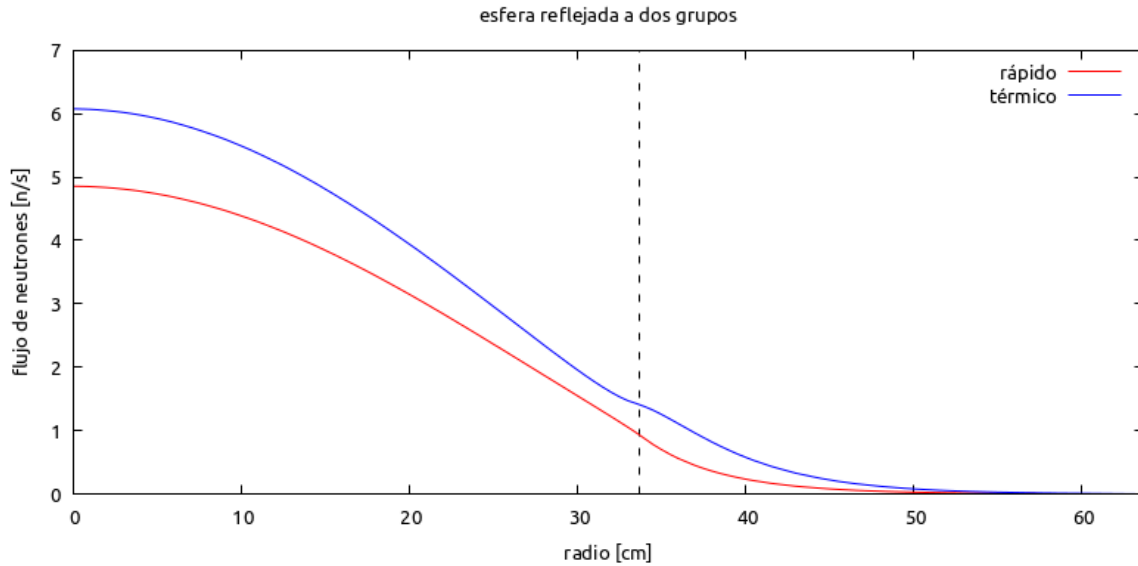
En la siguiente figura se muestra el resultado de la simulación realizada en una fracción de $1/32 \times 1/4$ de una esfera. Como se puede observar, la visualización realizada con el Gmsh presenta una visualización interesante sin realizar mucho esfuerzo.



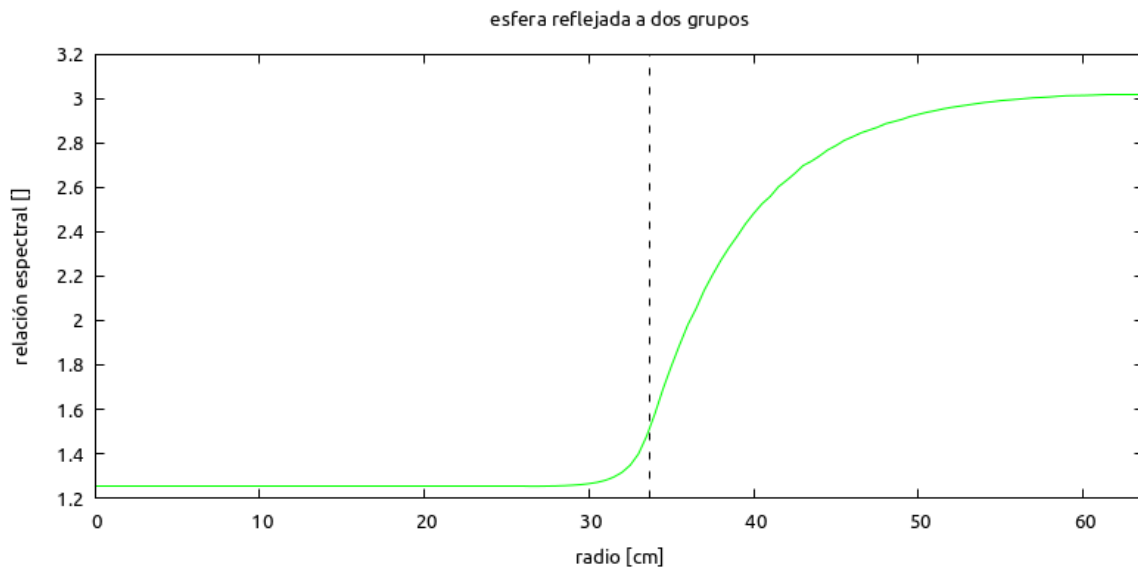
El problema que posee realizar gráficos 3D, es que resulta difícil mostrar múltiples variables al mismo tiempo. Por ello conviene mostrar gráficos 2D de un plano que se encuentre contenido en el dominio, como el mostrado en la próxima figura, la cual fue realizada con el GNUPlot, en base a un filtrado realizado con el FeenoX.



Pero como se dijo al principio, este problema es unidimensional en realidad, y por lo tanto lo mejor es extraer las distintas variables, en función del radio. Esta operación se puede realizar directamente en el FeenoX, y luego graficar el resultado con el GNUPlot, como se muestra en la siguiente figura.



Nuevamente en este ejemplo se puede apreciar que existe la región de transición, en donde la relación espectral deja de ser constante, y donde no es útil plantear la separabilidad de variables.



Cálculo de propiedades en base a los resultados

Para alguno de los ejercicios vamos a necesitar calcular algunas propiedades o funciones que no se obtienen por defecto como salida del FeenoX.

Un ejemplo sencillo es cuando se quiere reducir la dimensión de una función, como por ejemplo la distribución del flujo térmico. En este caso simplemente hay que definir una nueva función, de la siguiente manera:

```
phil_lineal(r) = phil(r*cos(angXY)*cos(angXZ), r*sin(angXY)*cos(angXZ), r*sin(angXZ))
```

Por otro lado, para poder realizar la condensación de propiedades, u obtener el factor de pico, es necesario realizar el promedio de las propiedades (secciones eficaces y coeficientes de difusión) en cada región que se quiere condensar, donde ese promedio es el pesado con el flujo o con su Laplaciano, según sea la propiedad:

$$\bar{\Sigma} = \frac{\int_E \int_{\text{Dominio}} \Sigma(E, \vec{r}) \cdot \phi(E, \vec{r}) \cdot d\vec{r} \cdot dE}{\int_E \int_{\text{Dominio}} \phi(E, \vec{r}) \cdot d\vec{r} \cdot dE} \quad \bar{D} = \frac{\int_E \int_{\text{Dominio}} D(E, \vec{r}) \cdot \nabla^2 \phi(E, \vec{r}) \cdot d\vec{r} \cdot dE}{\int_E \int_{\text{Dominio}} \nabla^2 \phi(E, \vec{r}) \cdot d\vec{r} \cdot dE}$$

Realizar este promedio tiene dos complicaciones de implementación en FeenoX:

- Definir el dominio de integración
- Calcular el Laplaciano

Por lo tanto, a continuación se muestran cómo realizar estos cálculos.

Laplaciano del flujo

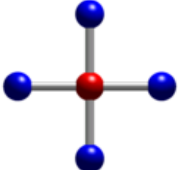
Dado que para poder conservar el factor de multiplicación es necesario conservar la tasa de fugas, o de derivas, es que al coeficiente de difusión hay que condensarlo pesando con el Laplaciano, y no con el flujo. Esto hace que sea necesario calcular el Laplaciano del flujo, para poder usarlo como factor de peso.

Como en el FeenoX esta distribución no está accesible para el usuario, es necesario que calcularlo manualmente. Para ello se puede utilizar la aproximación mediante diferencias finitas, mediante el algoritmo conocido como "[five-point stencil finite-difference](#)"



$$f''(x) \approx \frac{-f(x+2h) + 16f(x+h) - 30f(x) + 16f(x-h) - f(x-2h)}{12h^2}$$

Unidimensional



$$\nabla^2 f(x, y) \approx \frac{f(x-h, y) + f(x+h, y) + f(x, y-h) + f(x, y+h) - 4f(x, y)}{h^2}$$

Bidimensional

El problema con este cálculo es que el método numérico de resolución usado es el de elementos finitos, con lo cual la solución es interpolada linealmente entre los elementos, y por lo tanto la derivada segunda no está definida.

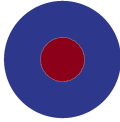
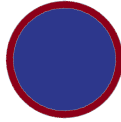
Sin embargo, si aún se desea estimar alguna derivada, es importante remarcar que obtener numéricamente una derivada suele tener mucho error, y con lo cual puede ser muy difícil obtener un cálculo decente de este parámetro. Para ello es necesario por un lado tener una malla lo suficientemente fina, como para tener una buena descripción espacial del flujo, y luego elegir adecuadamente la distancia de alejamiento para evaluar el cambio.

Propiedades promedio en una región

Para obtener el promedio de una propiedad en una región con el FeenoX, se puede hacer uso de la herramienta que realiza la integral numérica de una función calculada sobre una malla.

Sin embargo, esta herramienta realiza la integral sobre todo el dominio definido por una región, con lo cual es necesario utilizar máscaras, para filtrar la región de interés. Las máscaras son funciones que valen 0 o 1. Por ejemplo, si se desea dividir en 4 regiones a un reactor esférico reflejado, se podrían utilizar las siguientes máscaras:

- B1: Zona del núcleo lejos de la frontera
- B2: Zona del núcleo cerca de la frontera
- B3: Zona del reflector cerca de la frontera
- B4: Zona del reflector lejos de la frontera

				
máscara B1	máscara B2	máscara B3	máscara B4	máscara B5

Por otro lado, para la zona del reflector cercana a la frontera, lo formal sería hacer algo como la máscara B5. Sin embargo, dado que al ejecutar el comando de integración este solo lo resuelve para las celdas correspondientes al reflector, entonces las máscaras B5 y B3 son equivalentes.

Luego, el cálculo de la sección eficaz de absorción a un grupo, por zona, se realiza:

```

mascaraB1(x,y,z) = (sqrt(x^2+y^2+z^2)<radioHomoNuc)
phi(x,y,z) = phi1(x,y,z)+phi2(x,y,z)
INTEGRATE phi(x,y,z)*mascaraB1(x,y,z) OVER nucleo RESULT phiReg1
phiSa(x,y,z) = SigmaA_1(x,y,z)*phi1(x,y,z)+SigmaA_2(x,y,z)*phi2(x,y,z)
INTEGRATE phiSa(x,y,z)*mascaraB1(x,y,z)/phiReg1 OVER nucleo RESULT SaNuc1
    
```

Realizar simulaciones paramétricas

Para hacer variaciones de un parámetro, se puede ir realizando de a una simulación a la vez, o se puede automatizar el proceso, lo cual es lo recomendable.

Para realizar una corrida paramétrica variando, por ejemplo, la longitud característica, se puede ejecutar el siguiente comando:

```

echo "" | awk '{for(i=2;i<10;i++){system("feenox in_reactor.fee 40 "i" ")}exit;}' >
out_param-lc.dat
    
```

En este caso se está variando el parámetro con los números enteros comprendidos entre 2 y 9.

Otra opción, es generar la lista valores que se le quiere dar al parámetro, y guardarlo en un archivo de texto, con un número por renglón. Si este archivo con los valores se llama por ejemplo "00-listaNumeros.txt", entonces el comando a ejecutar es el siguiente

```

awk '{system("feenox in_reactor.fee 40 "$0" ")}' 00-listaNumeros.txt > out_param-lc.dat
    
```

Finalmente, los valores van a quedar guardados en el archivo *“out_param-lc.dat”*, y por lo tanto luego pueden ser graficados por ejemplo usando el GNUPlot:

```
gnuplot -p -e 'plot "out_param-lc.dat" u 2:3 w l'
```